Le Potager Solarpunk

Fiche de préparation à l'oral · Projet 2 Unity · Kerouann Touami-Causse · BUT2 Informatique

1. Le jeu en 30 secondes

Le joueur pilote un drone horticole au-dessus d'un potager urbain. Il plante des graines de blé sur des parcelles de terre, les arrose pour accélérer leur croissance, et doit les récolter au bon moment : trop tôt elles ne sont pas prêtes, trop tard elles éclatent. Chaque récolte fait monter le score ; la partie se termine quand l'objectif de récolte est atteint, et le meilleur score est conservé d'une session à l'autre.

2. Commandes

Action	Clavier / souris	Manette
Se déplacer	ZQSD / flèches	Stick gauche
Tourner / incliner la vue	Souris	Stick droit
Monter / descendre	Espace / Ctrl gauche	RB / LB
Planter	Clic gauche	Gâchette droite (RT)
Changer de graine	E / A ou molette	Croix directionnelle → / ←
Arroser	Clic droit (maintenu)	Gâchette gauche (LT)

Toutes les commandes passent par un seul asset **Input Actions** : chaque action (Move, Plant…) regroupe les touches clavier ET les boutons manette. Le code lit l'action, jamais une touche précise.

3. Architecture : un script = une responsabilité

Script	Posé sur	Responsabilité
DroneController	Drone	Déplacer le drone (relatif à la caméra), gérer l'altitude, rester dans le potager
CameraOrbit	Main Camera	Suivre le drone et tourner autour (souris / stick droit)
SeedPlanter	Drone	Planter une graine sous le drone si la place le permet
SoilPlot	Chaque parcelle	Marquer la terre cultivable, limiter le nombre de plantes
Plant	Prefab de la plante (Ble)	Croissance, arrosage, récolte par aspiration, éclatement
WaterSprayer	Drone	Jet de particules + arroser les plantes sous le drone
ScoreManager	GameManager	Compter les points et les récoltes par plante (Dictionary), prévenir par événements
PlantSelector	Drone	Liste des plantes disponibles, graine sélectionnée, changement avec les touches
PlantSelectorUI	HUD (Canvas)	Afficher la graine sélectionnée (nom, points, position)
HarvestUI	HUD (Canvas)	Afficher le score total et le nombre de récoltes par plante

Organisation : tout le contenu du jeu est dans **Assets/_Project**, rangé par type (Scripts, Prefabs, Materials, Scenes, Input, Audio, VFX, UI, Models, Animations). Tous les réglages (vitesses, durées, limites, prefabs) sont des champs **[SerializeField]** modifiables dans l'Inspector, avec une infobulle (Tooltip).

4. Concepts techniques : questions probables et réponses

Déplacement et commandes (jalon 1)

Q : Comment le même code marche-t-il au clavier et à la manette ?
Le script reçoit une `InputActionReference` (l'action *Move*) et lit `ReadValue<Vector2>()`. C'est l'Input System qui fusionne ZQSD, flèches et stick dans une seule valeur entre -1 et 1. Aucun script différent par périphérique.

Q : Pourquoi activer les actions dans OnEnable et les désactiver dans OnDisable ?
C'est le cycle de vie d'un composant Unity : l'action n'écoute les touches que lorsque le composant est actif. On se désabonne aussi des événements dans OnDisable, sinon une action pourrait appeler un objet détruit.

Q : Pourquoi multiplier par Time.deltaTime ?
Update est appelé une fois par image. Sans deltaTime, un PC à 144 FPS déplacerait le drone 2,4 fois plus vite qu'un PC à 60 FPS. Avec deltaTime, la vitesse s'exprime en unités par seconde.

Q : Pourquoi la souris n'utilise-t-elle PAS Time.deltaTime ?
La souris renvoie un déplacement en pixels depuis la dernière image : c'est déjà une quantité par image. Le stick renvoie une position (-1 à 1) qu'on transforme en vitesse : lui doit être multiplié par deltaTime.

Q : Pourquoi la caméra est-elle mise à jour dans LateUpdate ?
LateUpdate s'exécute après tous les Update. Le drone a donc déjà bougé quand la caméra se place : elle suit sa position finale, sans tremblement.

Q : Comment le drone reste-t-il dans le potager et au-dessus du sol ?
Après chaque déplacement, `Mathf.Clamp` borne X et Z entre les limites du terrain, et l'altitude entre une hauteur minimale (1 m) et maximale. Le drone n'a pas de collision physique avec le sol : il vole, on impose directement sa position.

Q : Pourquoi le drone est-il un objet vide avec un visuel en enfant ?
Le code déplace le parent ; les animations (jalon 4) agiront sur l'enfant. Ainsi animation et gameplay ne s'écrasent pas mutuellement (piste de la brique 4.1).

Plantation et croissance (jalon 2)

Q : Comment la graine apparaît-elle exactement sous le drone ?
Un `Physics.Raycast` est lancé vers le bas depuis le drone. La graine est instanciée au point d'impact (`hit.point`) : elle tombe sous le drone, pas devant ni à côté.

Q : Comment sait-on qu'on est au-dessus de la terre ?
Chaque parcelle porte un composant `SoilPlot`. Si le collider touché par le rayon n'en a pas (herbe), on ne plante pas. Un composant est plus sûr qu'un Tag : aucune chaîne de caractères à mal orthographier.

Q : Pourquoi un Prefab ?
La plante est créée des dizaines de fois avec `Instantiate`. Modifier le Prefab modifie toutes les futures plantes.

Q : Comment gérer plusieurs plantes en même temps ?
Une variable simple ne garde qu'un objet ; on stocke donc les plantes dans une `List<Plant>`. Avant de l'utiliser, `RemoveAll(p => p == null)` retire celles qui ont été détruites (récoltées ou éclatées). La liste sert à vérifier l'espacement minimum et le nombre maximum par parcelle.

Q : Comment la plante attend-elle sans bloquer le jeu ?
La croissance est une **coroutine** (`IEnumerator Start()`). À chaque image, elle ajoute `Time.deltaTime × growthSpeed` au temps écoulé, met à jour la taille, puis `yield return null` rend la main au moteur jusqu'à l'image suivante.

Q : Pourquoi un enum GrowthStage plutôt que des booléens ?
Une plante est dans un seul état à la fois : Seed, Ripe ou Overripe. Une seule variable d'état évite les combinaisons impossibles (isSeed et isRipe vrais en même temps). Seul l'état Ripe est récoltable.

Q : Pourquoi deux graines plantées à quelques secondes d'écart ne mûrissent-elles pas ensemble ?
Chaque plante a sa propre coroutine qui démarre à sa plantation, et une petite variation aléatoire (±15 %) est appliquée à ses durées.

Q : Comment la plante disparaît-elle ?
`Destroy(gameObject)` la retire de la scène et de la Hierarchy à la fin de l'étape « trop mûr ».

Arrosage, récolte et score (jalons 2 et 3)

Q : Comment une plante sait-elle qu'elle est arrosée ?
Tant que le bouton est maintenu (`IsPressed()`), le drone fait un `Physics.OverlapCapsule` : une colonne invisible sous lui. Il appelle `Water()` sur chaque plante trouvée. La plante note jusqu'à quand elle reste humide ; tant qu'elle l'est, son temps de croissance avance × `wateredGrowthMultiplier` (2,5 par défaut, réglable).

Q : Pourquoi IsPressed() pour arroser et l'événement performed pour planter ?
Planter est une action ponctuelle : un appui = une graine (performed). Arroser est continu : on vérifie à chaque image si le bouton est encore maintenu (IsPressed).

Q : Le drone vole haut et le blé est au sol : comment se touchent-ils ?
Le blé porte une Box Collider en Is Trigger très haute (9 m) : une colonne invisible qui monte jusqu'au plafond de vol du drone. Le drone a un Rigidbody en Is Kinematic (déplacé par le code, pas par la physique) : c'est nécessaire pour que Unity déclenche les événements de trigger.

Q : Pourquoi OnTriggerStay et pas OnTriggerEnter ?
Si le drone est déjà au-dessus quand le blé devient mûr, Enter a eu lieu trop tôt. Stay est appelé tant que le drone reste dans la zone. Et comme on ne stocke aucune référence au drone, il n'y a rien à vider dans OnTriggerExit : aucune erreur si le blé éclate pendant le survol.

Q : Comment le blé, créé pendant la partie, connaît-il le ScoreManager ?
Par **injection de dépendance** : juste après `Instantiate`, le SeedPlanter appelle `newPlant.Initialize(scoreManager)`. On ne peut pas glisser cette référence dans l'Inspector du Prefab, car le ScoreManager est dans la scène.

Q : Comment l'affichage du score est-il mis à jour ?
Le ScoreManager déclenche un événement `ScoreChanged` ; le ScoreUI s'y abonne dans OnEnable et se désabonne dans OnDisable. Le ScoreManager ne connaît pas l'interface : chaque script garde une seule responsabilité.

Q : Comment l'interface reste-t-elle à sa place en 16:9, large et 4:3 ?
Le Canvas a un Canvas Scaler en Scale With Screen Size (référence 1920×1080, Match 0,5). Le texte est ancré en haut à gauche avec un pivot (0, 1) : il reste collé au coin quelle que soit la taille de l'écran. Un contour (Outline) le garde lisible sur le décor.

Q : Comment fonctionne l'aspiration à la récolte ?
Les points sont donnés immédiatement (pas de délai visible), puis `StopAllCoroutines()` arrête la croissance et une coroutine fait monter le blé vers la position actuelle du drone en tournant et en rétrécissant (interpolation `Vector3.Lerp` avec une accélération t^2), avant `Destroy`.

Plusieurs plantes et affichage (bonus « Légumes inégaux »)

Q : Comment chaque plante a-t-elle ses propres règles ?
Tout est défini sur le Prefab de la plante (composant Plant) : nom affiché, durées de pousse, points, nombre maximum par parcelle, espacement. Ajouter une plante = créer un Prefab, sans toucher au code. La parcelle demande simplement à la plante s'il reste de la place (HasRoomFor).

Q : Comment la pousse est-elle rendue réaliste ?
La plante affiche une liste de modèles de pousse (ex. Wheat_1 puis Wheat_2) un par un selon l'avancement, puis le modèle mûr, puis le modèle tombé (trop mûr). Un seul modèle est visible à la fois (SetActive).

Q : Comment change-t-on de graine ?
Le PlantSelector garde un tableau de Prefabs (`Plant[]`) et un index. Les actions NextPlant / PreviousPlant changent l'index en bouclant avec un modulo ; un événement `SelectionChanged` prévient l'interface. Le SeedPlanter plante `plantSelector.CurrentPlant`.

Q : Comment compte-t-on les récoltes par plante ?
Le ScoreManager utilise un `Dictionary<string, int>` : clé = nom de la plante, valeur = quantité. HarvestUI s'abonne aux événements et reconstruit le texte avec un StringBuilder.

5. Choix personnels à savoir justifier

- **Caméra orbitale** (souris / stick droit) au lieu d'une caméra fixe : elle démarre en vue de dessus (90°) comme demandé, mais le joueur peut tourner et incliner la vue. Le déplacement reste intuitif car il est relatif à la caméra : « haut » = haut de l'écran.
- **Altitude contrôlable** (Espace / Ctrl, RB / LB). ⚠ La brique 1.1 demande une hauteur constante : justification = le drone ne s'enfonce jamais (plancher à 1 m, plafond à 8 m) et la hauteur reste bornée et maîtrisée. Point de vigilance pour la récolte : la zone de détection des légumes doit monter jusqu'au plafond.
- **Parcelles** : on ne plante que sur la terre, avec un nombre maximum de plantes par parcelle et un espacement minimum, pour un potager lisible et une vraie gestion de l'espace.
- **Thème** : le blé, avec trois visuels distincts (graine, blé mûr, blé trop mûr). Durées : 10 s pour mûrir, 7 s de fenêtre de récolte, 4 s avant d'éclater.
- **Récolte par aspiration** : au lieu de disparaître d'un coup, le blé s'envole en tourbillonnant vers le drone (proche du défi bonus « Juiciness »).
- **Commandes manette** : stick gauche = déplacement, stick droit = caméra, gâchettes = planter (RT) / arroser (LT), boutons de tranche = altitude.
- **Plusieurs plantes** (blé, carotte…) sélectionnables avec affichage à l'écran, chacune avec ses durées, ses points et sa capacité par parcelle → défi bonus « Légumes inégaux ».
- **Pousse réaliste** : vrais modèles 3D par stade au lieu d'un objet qui grossit.

6. Avancement

Jalon	Contenu	État
1	Le drone survole le potager	Terminé
2	Planter (2.1) · Pousser (2.2) · Arroser (2.3)	Terminé
3	Récolte (trigger, aspiration) · Score à l'écran	Terminé
4	Animations · Sons et particules · Décor solarpunk	Vrais modèles de plantes en place · reste à faire
Bonus	Plusieurs plantes (Légumes inégaux) · récolte par aspiration	Fait
5	Écran titre / fin / pause · Meilleur score (PlayerPrefs)	À faire
6	Rangement · Debug · Build et itch.io · Dépôt Git	Dépôt GitHub en ligne · rangement commencé

Dépôt de code : https://github.com/kerker48/Solarpunk · à mettre à jour au fil du projet.

7. Crédits des assets

Asset	Auteur	Licence	Utilisé pour
Ultimate Crops Pack	Quaternius (quaternius.com · patreon.com/quaternius)	CC0 1.0 Universal (domaine public)	Le blé à ses différents stades (Wheat_1 à Wheat_4, Wheat_Crop)
Stylized Nature MegaKit (version Standard gratuite)	Quaternius (quaternius.com)	CC0 1.0 Universal (domaine public)	Décor végétal : arbres, buissons, herbes, fleurs, cailloux
Particle Pack (prévu)	Kenney (kenney.nl)	CC0 (à vérifier dans le fichier License du pack)	Particules d'eau, de récolte et d'éclatement

Source : fichiers License.txt et License_Standard.txt fournis avec les packs. La licence CC0 n'oblige pas à créditer, mais la fiche du projet l'exige : ces crédits doivent figurer sur la page itch.io (brique 6.3). Formulation possible : « Modèles 3D : Ultimate Crops Pack et Stylized Nature MegaKit par Quaternius (CC0) ».

8. Déroulé conseillé de la démonstration

- Montrer le projet : dossier **_Project** rangé, Hierarchy lisible, Console vide.
- Dans l'Inspector du Drone : vitesse, altitude min/max ; sur le Prefab Plant : les durées de croissance (moins de 30 s pour les trouver).
- Play Mode : déplacement clavier puis manette, rotation de la caméra, montée / descente bloquée à 1 m.
- Planter sur une parcelle, essayer sur l'herbe (refus), essayer trop près (refus), remplir une parcelle (refus au-delà du maximum).
- Laisser une plante passer par les 3 étapes et éclater : elle disparaît de la Hierarchy.
- Arroser une graine sur deux plantées côte à côte : l'arrosée mûrit la première.
- Survoler une graine (rien), puis un blé mûr : il est aspiré, le score monte. Changer le format de la vue Game : le score reste en haut à gauche.
- Suite (quand ce sera fait) : animations, sons, décor, écrans, build itch.io.